Heterogeneous Computing Lab
GPU Tutorial (Part 1/3)

Learning Objectives and Organization

Learning Objectives: (1) Understand for what classes of applications are GPUs are inherently suited for; (2) Have practical experience with running and writing code that runs on GPUs; (3) Evaluate performance between running highly parallelizable code on GPUs and CPUs.

Organization: The current document is the first out of the three documents provided for this lab, where the main goal is for you get familiarized and have some intial experience in working with GPUs. After this assignment, in the second week, we will take a step back and look at the architectural characteristics that enable GPUs to perform well on highly paralellizable tasks.

1 Jumping Right-in: CPU vs GPU in the Context Image Processing

GPUs excels at solving embarrassingly parallel problems (problems with no communication between threads, and no perfect execution order). The perfect example for the power of such hardware is the example of modern image and video processing tasks. As such, the main idea for this assignment is not to build a complicated image processing system, but to observe what happens when the same computation must be repeated many times over many pixels and many frames. In this practical setting, the difference between a CPU and a GPU becomes very clear.

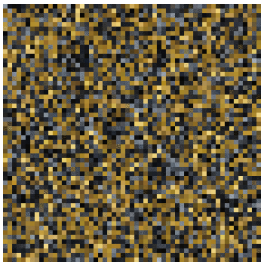
You will be given a sequence of obfuscated image frames extracted from a video, together with a small metadata file and a manifest describing how the frames were transformed. These transformations are deterministic and reversible. Your task is to reconstruct the original frame sequence by implementing the inverse of those transformations in a partially completed Python file provided in the repository.

1.1 What We Have Done So Far (Tuesday, 7th April, 2026)

. During the first lab, you have had the opportunity to see the idea of the pipeline in action. Specifically, you were provided with a code repository containing a small CPU demonstration that operated on three frames.



(a) Initial photo



(b) Asset for students



(c) Reconstructed photo

Figure 1: An example frame before transformation (during the first lab, we tested using a video of a swan), the obfuscated frame provided to you, and the reconstructed result after running the inverse pipeline.

This reduced example was meant to show that even when the input is very small, repeatedly permuting pixels and tiles already leads to noticeable runtime on a CPU. Subsequently, you have seen how this same basic type of computation can then be executed much more efficiently on a GPU, where many operations can be carried out in parallel. Each image was processed independently through a sequence of deterministic transformations. These transformations are applied repeatedly across multiple spatial scales and for many iterations. Although each individual operation is simple, the total amount of work becomes substantial due to the number of repetitions and the number of pixels involved. To make this computational cost explicit, we measured with the script the time required to process each frame. During execution, you observed output of the form:

```
[CPU] forward frame 1/3 ... done in 2.31s
[CPU] forward frame 2/3 ... done in 2.28s
[CPU] forward frame 3/3 ... done in 2.35s
```

The main takeaway from this demonstration is that, although the transformation itself is simple and perfectly reversible, its repeated application over large image data leads to significant computational cost on a CPU. The workload is inherently parallel, but the CPU implementation does not exploit this structure.

1.2 What you need to do next (Thursday, 7th April, 2026 AND Tuesday, 14th April, 2026)

Differently to the experience of the first lab, you will now be provided directly with a set of frames without knowing the original content of the images. Your task, similar to what you experienced during the first session, is to reconstruct the original frames by implementing the inverse of a transformation pipeline that has already been applied to them. After reconstruction, you should combine the frames into a video in order to recover the original signal.

More specifically, the exercise is based on a reversible image-processing pipeline implemented in PyTorch. Before looking into the underlying functionality in more detail, it is useful to note that this setup provides a first layer of abstraction over direct GPU programming. In practice, high-level languages such as Python, together with libraries like PyTorch and CUDA, make it possible to access GPU computing without having to deal with the low-level details of hardware interfacing. This was one of the key factors behind the rapid development of modern AI, as it enabled researchers and engineers to write efficient code much more quickly.

An additional advantage of this abstraction is that the same code can run either on a CPU or on a CUDA-enabled GPU. As such, the main purpose of this exercise is to demonstrate that a sequence of simple, exactly invertible transformations can be composed into a computationally intensive pipeline. Although the reconstruction process is conceptually straightforward, it becomes significantly more expensive when executed on a CPU compared to a GPU. This highlights how access to software toolkits such as CUDA, which enable computations to be executed automatically on GPUs when available, can lead to substantial performance improvements.

Concretely, in the associated repository (https://github.com/spetrescu/cpu-gpu-example.git) will be provided with:

- a folder containing already transformed (obfuscated) frames (`frames/`)
- a transformation recipe describing how the forward pass was constructed
- the full implementation of the forward pipeline (`forward_batch.py`)
- a partially completed reconstruction file containing `TODO` sections (**inverse_batch_exercise.py**)

Your task is to complete the missing parts of the reconstruction code.

1.3 Forward Pipeline

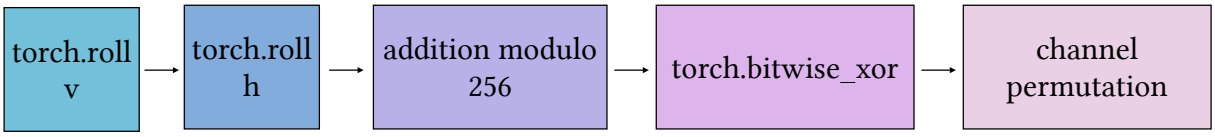


Figure 2: Transformations applied.

The forward pipeline has already been implemented for you - it processes images in batches using PyTorch tensors of shape `(N, H, W, 3)`. Each image is passed through a sequence of stages, and each stage consists of several repeated substeps.

At a high level, each substep applies the following operations in the order showcased in Figure 2. Concretely, the forward implementation makes use of the following functions:

- `torch.roll(..., dims=1)` for vertical shifts,
- `torch.roll(..., dims=2)` for horizontal shifts,
- `torch.bitwise_xor(...)` together with a mask generated by `make_mask(...)` ,

- integer addition modulo 256 using casting (e.g. `to(torch.int16)` and back),
- channel permutations using indexing such as `x[..., [1,2,0]]`.

All of these operations are deterministic and reversible. The transformed images are saved to disk after the forward pass, and these are the inputs you will work with.

1.4 Reconstruction Task (your implementation)

Among the files that you are given in the associated repository, your goal is to complete the `TODO` sections in `inverse_batch_exercise.py`. Practically, to reconstruct the original images, you must apply the same operations as in the forward pass, but in reverse order, and replace each operation with its inverse. This means that (1) the sequence of stages must be traversed in reverse, (2) within each stage, the substeps must also be processed in reverse, (3) within each substep, the operations must be undone in reverse order. As such, inside each `TODO` block, you will need to implement the inverse of the following operations:

- Undo the channel permutation (inverse of indexing such as `x[..., [1,2,0]]`),
- Undo the modular addition (reverse of + modulo 256),
- Undo the XOR operation using `torch.bitwise_xor(...)` with the same mask,
- Undo the spatial shifts using `torch.roll(...)` with opposite shift values.

The function `make_mask(...)` is already provided and must be reused exactly as in the forward pass.

1.5 CPU vs GPU Execution

The reconstruction code can be executed on both CPU and GPU. This is controlled by the `--device` argument, which can be set to either `cpu` or `cuda`. As you are provided with both versions of the exercise, it is important for you to remember that (1) in the CUDA version, where tensors are moved using `.to("cuda")` and batches are processed on the GPU, and in the (2) CPU version, where the same code runs without CUDA (i.e. no device transfer).

The logic of the reconstruction is identical in both cases – the only difference is the device on which the tensors are stored and processed. In other words, the CPU version is not a different algorithm; it is simply the same implementation executed without GPU acceleration.

1.6 What you should observe

Although each individual operation (e.g. `torch.roll` or `torch.bitwise_xor`) is simple, the pipeline applies these operations many times over many pixels and many frames. When combined with batching, this results in a large amount of total work. On the GPU, these operations are executed in parallel across pixels and images, which leads to significant speedups. On the CPU, the same operations cannot be as performed efficiently, making the reconstruction process much slower as the number of stages and frames increases.

1.7 Final goal

After completing the `TODO` sections, your implementation should:

- correctly reconstruct the original frames
- allow you to compare runtime between CPU and GPU execution.

Also, you should generate plots comparing CPU and GPU runtime and analyze how performance scales with batch size. This will provide a clear demonstration of why GPUs are particularly well-suited for workloads involving many repeated, independent operations over large datasets. Finally, you should create a montage of an mp4 with the sequence of images – you can choose 24fps.